BIKE STORE RELATIONAL DATABASE

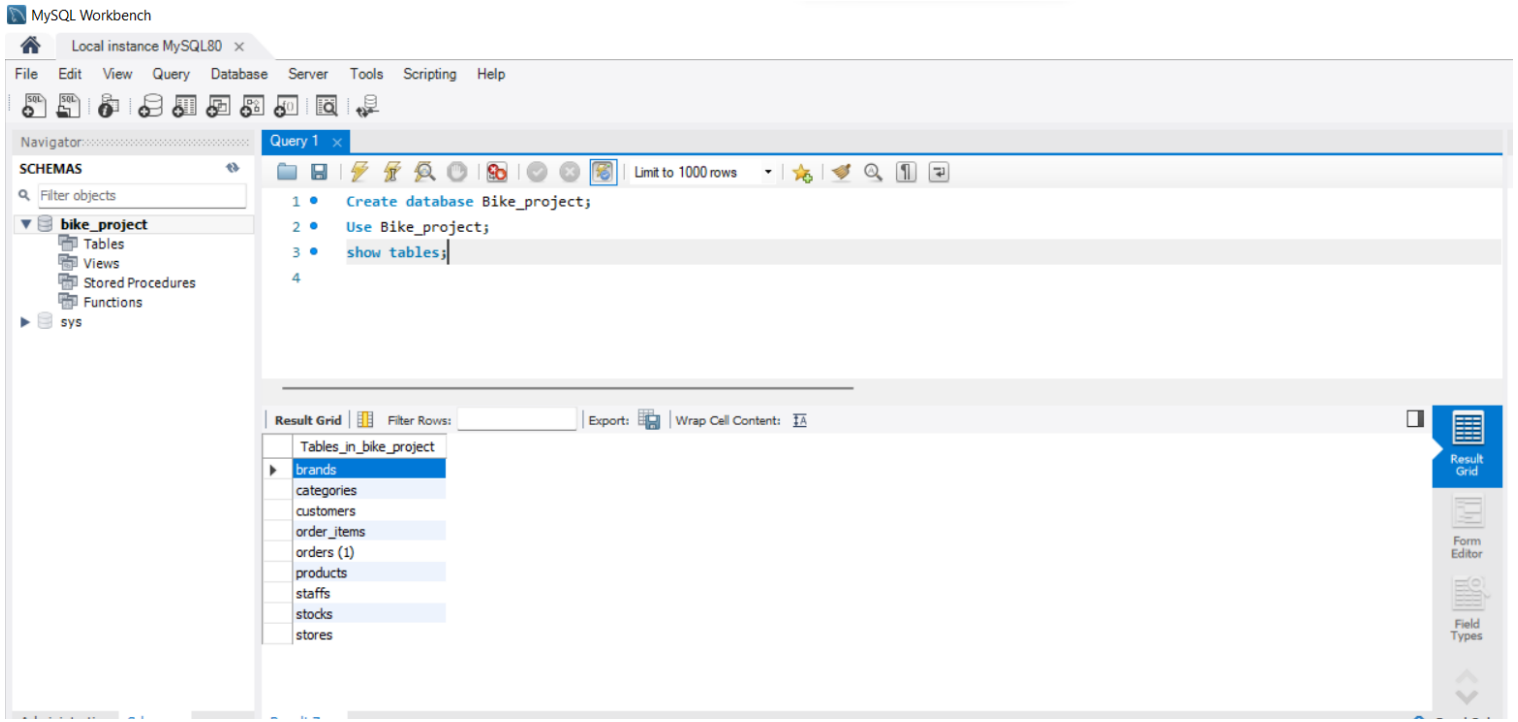
SQL PROJECT PRESENTATION

BY :- YOGAPRIYA RAJA DEVENDRA

* INTRODUCTION

- The Bike Store Relational Database is a SQL project used to manage and analyse data related to a bike store.
- - The database contains information about products, brands, customers, orders, and stores.
- - The data is organised into nine tables.
- - SQL is used to retrieve, filter, sort, and analyse the data.
- - MySQL Server and MySQL Workbench are used to perform SQL queries.

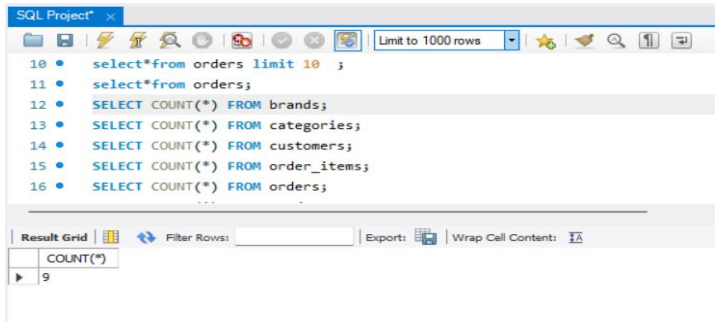
*Dataset and Tables

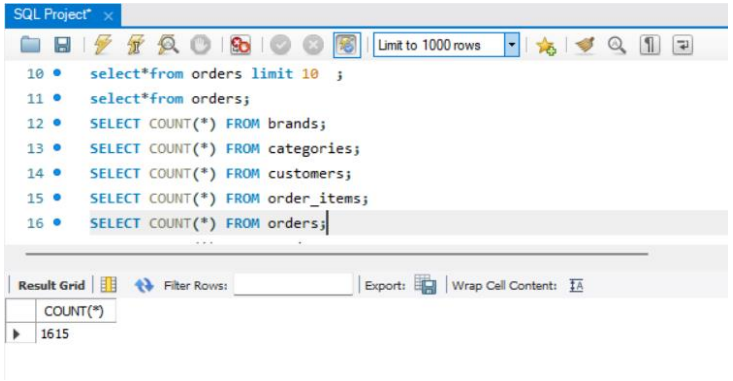


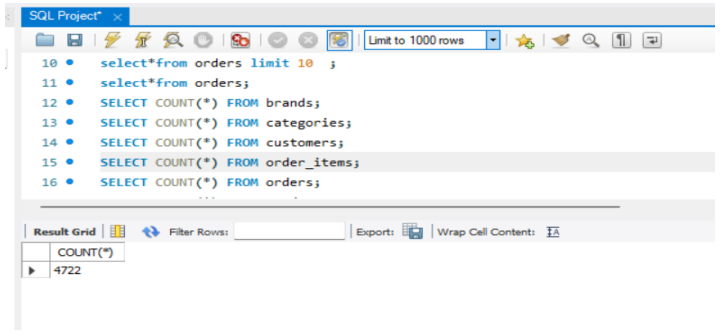
*SQl Queries and implementation

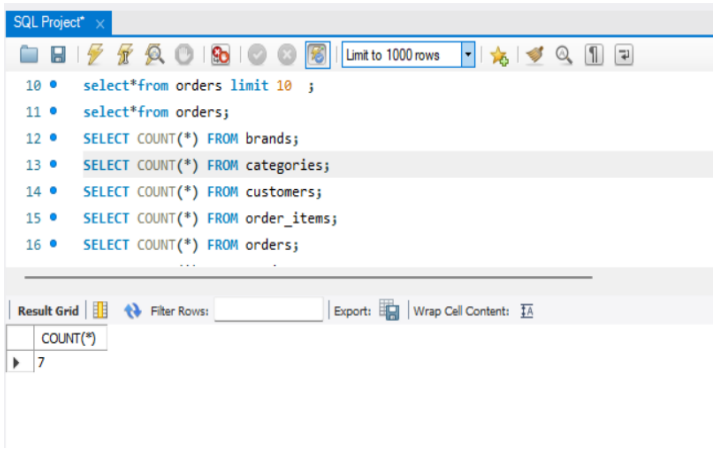
- The following operations were performed:-
- Displaying all tables using "SHOW TABLES".
- Counting records in each table.
- Filtering products using the "WHERE" clause.
- Sorting products using "ORDER BY".
- Grouping products using "GROUP BY".
- Joining tables to retrieve related information.
- Using subqueries to analyse product prices.

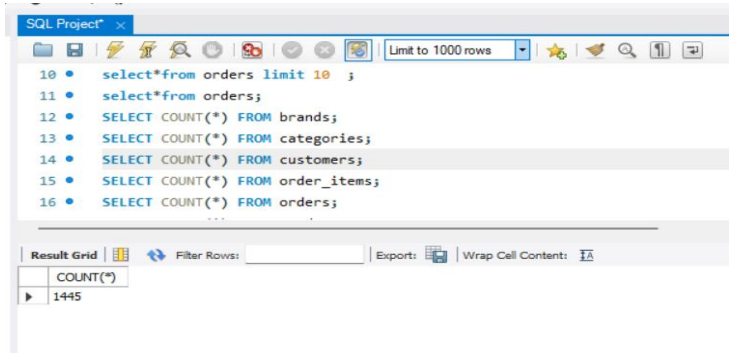
*Total count of each table

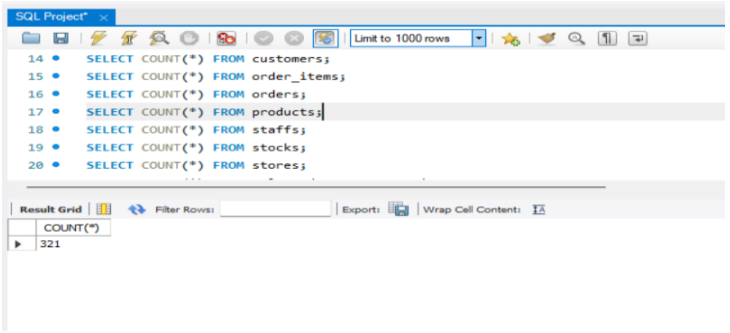




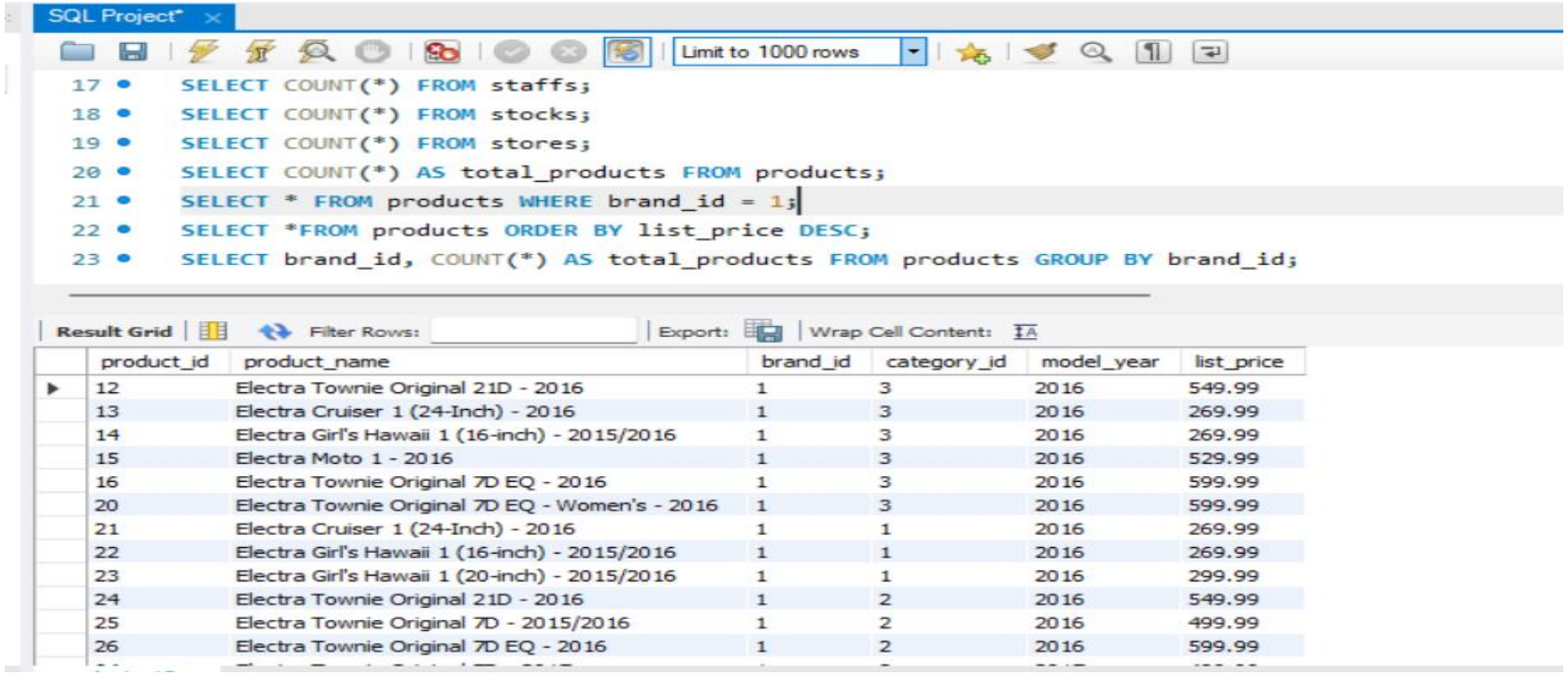








*Where :- filters products based on specific condition



product_id	product_name	brand_id	category_id	model_year	list_price
12	Electra Townie Original 21D - 2016	1	3	2016	549.99
13	Electra Cruiser 1 (24-Inch) - 2016	1	3	2016	269.99
14	Electra Girl's Hawaii 1 (16-inch) - 2015/2016	1	3	2016	269.99
15	Electra Moto 1 - 2016	1	3	2016	529.99
16	Electra Townie Original 7D EQ - 2016	1	3	2016	599.99
20	Electra Townie Original 7D EQ - Women's - 2016	1	3	2016	599.99
21	Electra Cruiser 1 (24-Inch) - 2016	1	1	2016	269.99
22	Electra Girl's Hawaii 1 (16-inch) - 2015/2016	1	1	2016	269.99
23	Electra Girl's Hawaii 1 (20-inch) - 2015/2016	1	1	2016	299.99
24	Electra Townie Original 21D - 2016	1	2	2016	549.99
25	Electra Townie Original 7D - 2015/2016	1	2	2016	499.99
26	Electra Townie Original 7D EQ - 2016	1	2	2016	599.99

Order by :- sorts products by price in descending order.

SQL Project*

Limit to 1000 rows

```
17 •  SELECT COUNT(*) FROM staffs;
18 •  SELECT COUNT(*) FROM stocks;
19 •  SELECT COUNT(*) FROM stores;
20 •  SELECT COUNT(*) AS total_products FROM products;
21 •  SELECT * FROM products WHERE brand_id = 1;
22 •  SELECT *FROM products ORDER BY list_price DESC;
23 •  SELECT brand_id, COUNT(*) AS total_products FROM products GROUP BY brand_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	brand_id	category_id	model_year	list_price
155	Trek Domane SLR 9 Disc - 2018	9	7	2018	11999.99
149	Trek Domane SLR 8 Disc - 2018	9	7	2018	7499.99
51	Trek Silque SLR 8 Women's - 2017	9	7	2017	6499.99
156	Trek Domane SL Frameset - 2018	9	7	2018	6499.99
157	Trek Domane SL Frameset Women's - 2018	9	7	2018	6499.99
169	Trek Emonda SLR 8 - 2018	9	7	2018	6499.99
50	Trek Silque SLR 7 Women's - 2017	9	7	2017	5999.99
56	Trek Domane SLR 6 Disc - 2017	9	7	2017	5499.99
148	Trek Domane SL 8 Disc - 2018	9	7	2018	5499.99
154	Trek Domane SLR 6 Disc Women's - 2018	9	7	2018	5499.99
177	Trek Domane SLR 6 Disc - 2018	9	7	2018	5499.99
43	Trek Fuel EX 9.8 27.5 Plus - 2017	9	6	2017	5299.99

products 14

Group By :- counts the number of products for each brand.

SQL Project*

Limit to 1000 rows

```
20    SELECT COUNT(*) FROM stocks;
21    SELECT COUNT(*) FROM stores;
22    SELECT COUNT(*) AS total_products FROM products;
23    SELECT * FROM products WHERE brand_id = 1;
24    SELECT *FROM products ORDER BY list_price DESC;
25    SELECT brand_id, COUNT(*) AS total_products FROM products GROUP BY brand_id;
26    SELECT
```

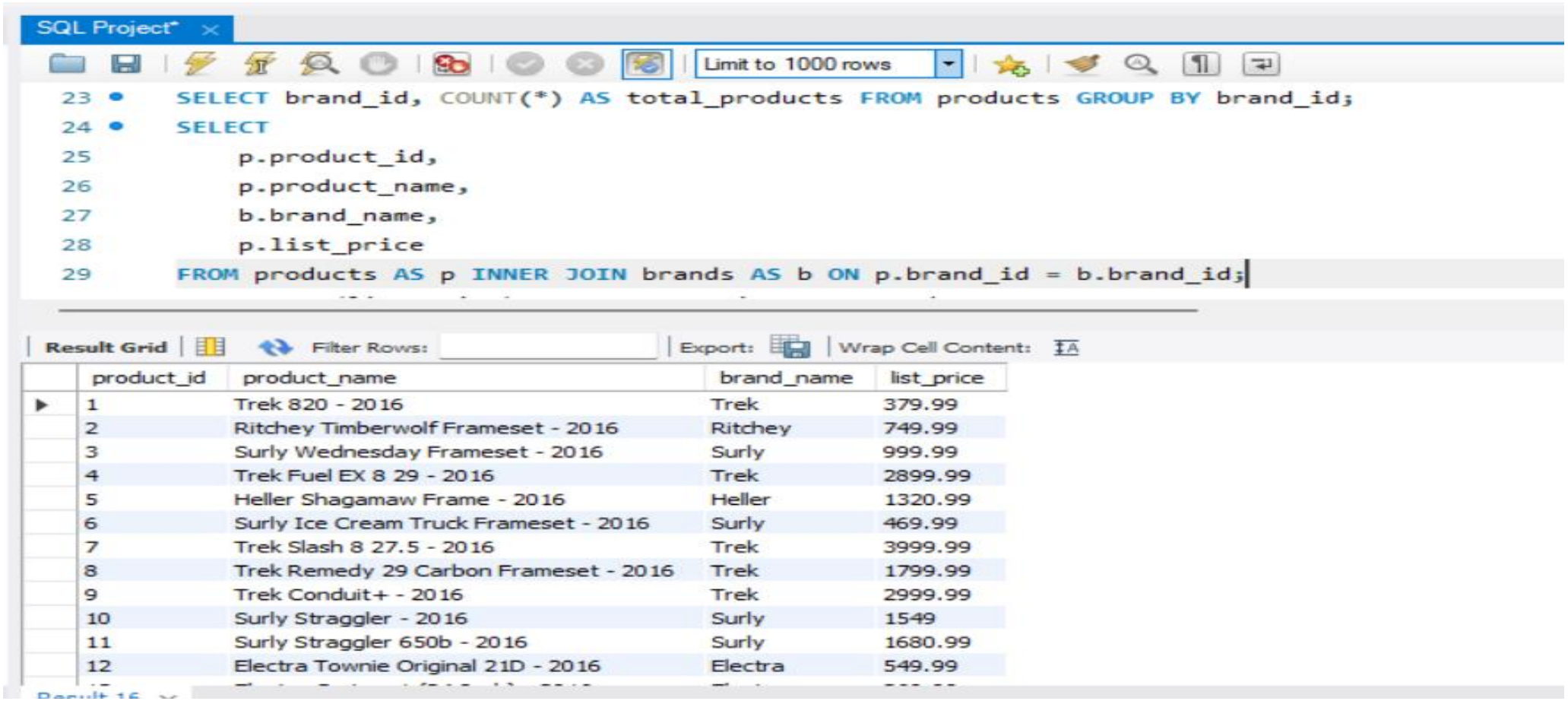
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

brand_id	total_products
9	135
5	1
8	25
3	3
1	118
4	3
7	23
2	10
6	3

*SQL Joins

- SQL JOIN is used to combine data from two or more tables based on a related column.
- JOIN Used in the Project
- Combined the Brands and Products tables.
- Matched records using "brand_id".
- Displayed brand names along with product names.
- JOIN helps retrieve related information stored across different tables in the database.

*Inner join:- Returns only the matching records from both tables.



*Left join:- Returns all records from the left table and matching records from the right table.

SQL Project*

Limit to 1000 rows

```
38          o.order_id,
39          o.order_date
40      FROM customers AS c INNER JOIN orders AS o ON c.customer_id = o.customer_id;
41      SELECT LEFT('BICYCLE', 3) AS result;
42      SELECT  RIGHT('BICYCLE', 3) AS result;
43      SELECT b.brand_name, p.product_name
44      FROM brands AS b LEFT JOIN products AS p ON b.brand_id = p.brand_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

brand_name	product_name
Electra	Electra Townie Original 21D Ladies' - 2018
Electra	Electra Townie Original 21D EQ Ladies' - 2018
Electra	Electra Townie Original 1 Ladies' - 2018
Electra	Electra Townie Commute 8D Ladies' - 2018
Electra	Electra Townie Commute 8D - 2018
Electra	Electra Townie Commute 27D Ladies - 2018
Electra	Electra Townie Commute 27D - 2018
Electra	Electra Townie Balloon 8D EQ Ladies' - 2016/20...
Electra	Electra Townie Balloon 8D EQ - 2016/2017/2018
Electra	Electra Townie Balloon 7i EQ Ladies' - 2017/2018
Electra	Electra Townie Balloon 3i EQ - 2017/2018
Electra	Electra Townie Original 21D EQ - 2017/2018

Right join:- Returns all records from the right table and matching records from the left table.

SQL Project*

Limit to 1000 rows

```
40      FROM customers AS c INNER JOIN orders AS o ON c.customer_id = o.customer_id;
41 •    SELECT LEFT('BICYCLE', 3) AS result;
42 •    SELECT  RIGHT('BICYCLE', 3) AS result;
43 •    SELECT b.brand_name, p.product_name
44      FROM brands AS b LEFT JOIN products AS p ON b.brand_id = p.brand_id;
45 •    SELECT b.brand_name, p.product_name FROM products AS p
46      RIGHT JOIN brands AS b ON p.brand_id = b.brand_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

brand_name	product_name
Electra	Electra Townie Original 21D Ladies' - 2018
Electra	Electra Townie Original 21D EQ Ladies' - 2018
Electra	Electra Townie Original 1 Ladies' - 2018
Electra	Electra Townie Commute 8D Ladies' - 2018
Electra	Electra Townie Commute 8D - 2018
Electra	Electra Townie Commute 27D Ladies - 2018
Electra	Electra Townie Commute 27D - 2018
Electra	Electra Townie Balloon 8D EQ Ladies' - 2016/20...
Electra	Electra Townie Balloon 8D EQ - 2016/2017/2018
Electra	Electra Townie Balloon 7i EQ Ladies' - 2017/2018
Electra	Electra Townie Balloon 3i EQ - 2017/2018
Electra	Electra Townie Original 21D EQ - 2017/2018

*Subqueries

- A subquery is a query written inside another SQL query. It helps retrieve data based on the result of an inner query.
- Subqueries Used in the Project
- Identified products with prices above the average.
- Identified products with prices below the average.
- Found the highest-priced products.
- Retrieved customers who have not placed orders.
- Subqueries help perform more advanced data analysis using SQL.

*Subqueries:-

SQL Project*

Limit to 1000 rows

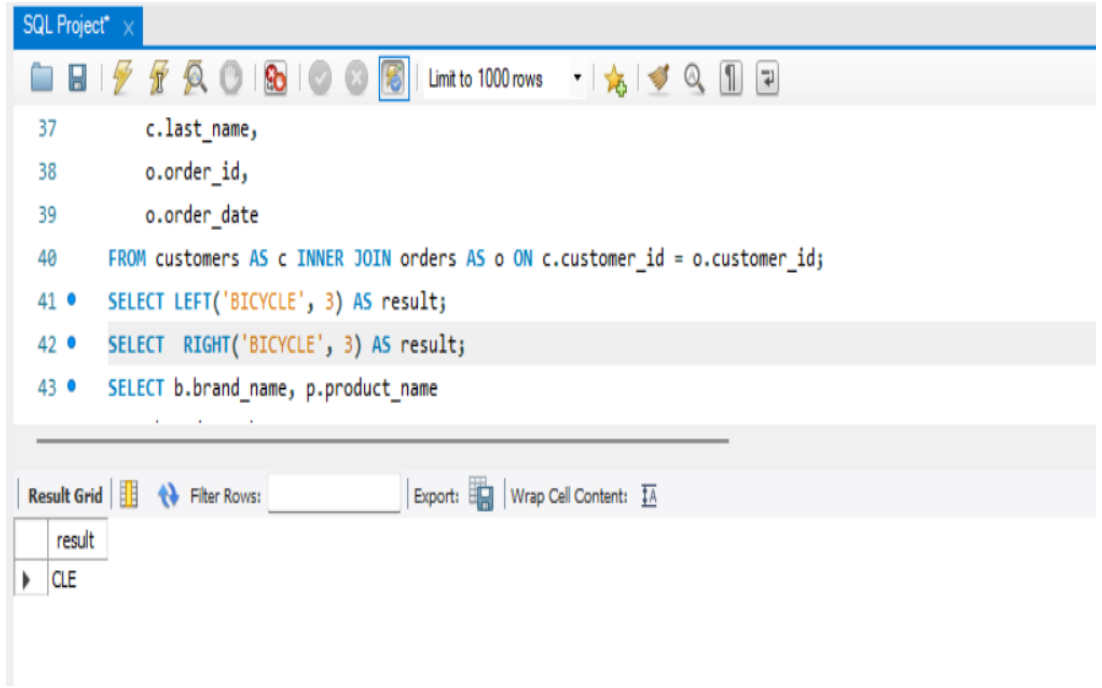
```
42 •  SELECT  RIGHT('BICYCLE', 3) AS result;
43 •  SELECT b.brand_name, p.product_name
44    FROM brands AS b LEFT JOIN products AS p ON b.brand_id = p.brand_id;
45 •  SELECT b.brand_name, p.product_name FROM products AS p
46    RIGHT JOIN brands AS b ON p.brand_id = b.brand_id;
47 •  SELECT product_name, list_price FROM products
48    WHERE list_price < (SELECT AVG(list_price) FROM products);
```

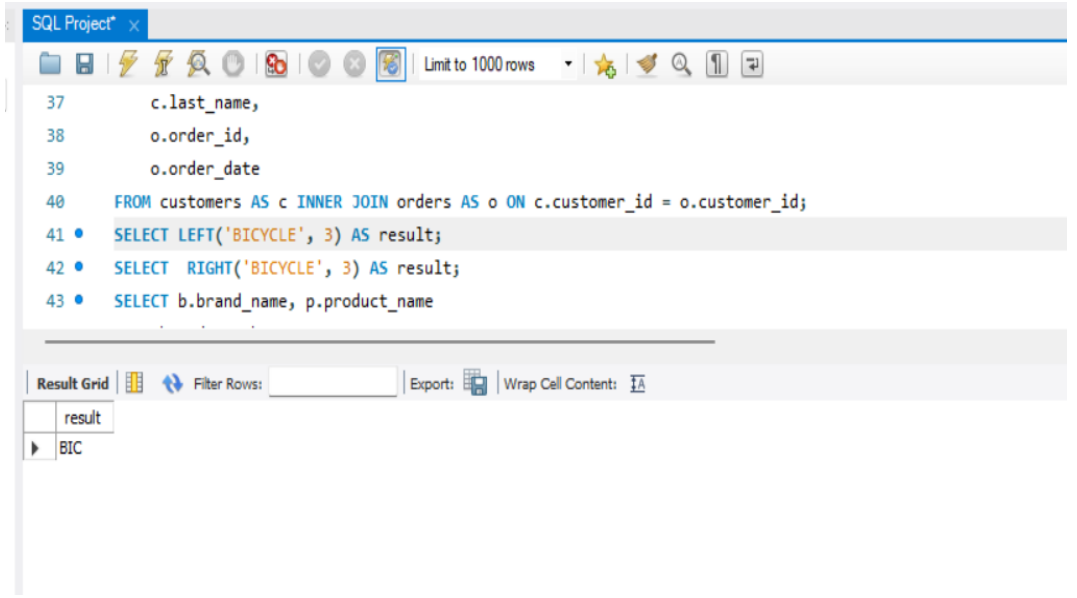
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	list_price
Trek 820 - 2016	379.99
Ritchey Timberwolf Frameset - 2016	749.99
Surly Wednesday Frameset - 2016	999.99
Heller Shagamaw Frame - 2016	1320.99
Surly Ice Cream Truck Frameset - 2016	469.99
Electra Townie Original 21D - 2016	549.99
Electra Cruiser 1 (24-Inch) - 2016	269.99
Electra Girl's Hawaii 1 (16-inch) - 2015/2016	269.99
Electra Moto 1 - 2016	529.99
Electra Townie Original 7D EQ - 2016	599.99
Pure Cycles Vine 8-Speed - 2016	429
Pure Cycles Western 3-Speed - Women's - ...	449

products 24

*String Functions:-





*Aggregate Functions:-

SQL Project*

Limit to 1000 rows

```
26          p.product_name,
27          b.brand_name,
28          p.list_price
29      FROM products AS p INNER JOIN brands AS b ON p.brand_id = b.brand_id;
30      SELECT AVG(list_price) AS average_price FROM products;
31      SELECT min(list_price)  FROM products;
32      SELECT max(list_price)  FROM products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

max(list_price)
11999.99

SQL Project*

Limit to 1000 rows

```
25          p.product_id,
26          p.product_name,
27          b.brand_name,
28          p.list_price
29      FROM products AS p INNER JOIN brands AS b ON p.brand_id = b.brand_id;
30      SELECT AVG(list_price) AS average_price FROM products;
31      SELECT min(list_price)  FROM products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

min(list_price)
89.99

SQL Project*

Limit to 1000 rows

```
25          p.product_id,
26          p.product_name,
27          b.brand_name,
28          p.list_price
29      FROM products AS p INNER JOIN brands AS b ON p.brand_id = b.brand_id;
30      SELECT AVG(list_price) AS average_price FROM products;
31      SELECT order_status, COUNT(*) AS total_orders FROM orders GROUP BY order_status;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

average_price
1520.5914018691537

*Advantages:-

- Organizes data into related tables.
- Makes data retrieval faster and easier.
- Helps filter and sort product information.
- Makes it easy to analyse product prices and brands.
- Reduces manual work and improves data management.
- Helps identify useful information using SQL queries.

Conclusion:-

- The Bike Store Relational Database project helped me understand how to use SQL to manage and analyse data. I learned to retrieve, filter, and sort data using SQL queries. I also practised aggregate functions, JOINs, and subqueries using MySQL Workbench. This project improved my practical knowledge of SQL and relational databases.
- Thank You